

Benchmarking and Performance Analysis of LLM Inferencing Systems

Saksham Rathi

Systems for LLMs
Under Prof. Mythili Vutukuru

Indian Institute of Technology Bombay
Autumn 2025-26

Contents



1. Various Scheduler Designs
   - vLLM Schedulers
   - Our Schedulers
   - Already Existing Schedulers
2. Throughput Numbers

First Come First Serve Policy



- Adds requests one by one, from the core engine into a waiting queue
- The requests are extracted in a first come first serve manner
- We stop selecting requests from the waiting queue into the running batch, once the size reaches `max_batch_size`

Priority Based



- The user passes priority values along with the requests' strings themselves
- We use a min heap to find the request with the smallest priority value (executed at first)
- Ties are broken in a first come first serve manner (through tracking request's arrival time)

Greedy First Come First Serve



- We keep a track of the working set along with the requests we add to the current running scheduler batch
- We add requests in a first come first serve manner
- However, now we are also controlling the batch size - we stop whenever addition of the next request increases the working set size beyond a threshold (decided by L2 cache size)
- For our workload, and GPU - it boils down to stopping whenever the next request belongs to a different family

Greedy First Come First Serve - Appendix - Expected Batch Size



Let's say we have N families. The user passes a bunch of requests, which can belong to any of these families. Moreover, all the requests have been shuffled, such that the probability of a request at position i to be of family j is exactly $\frac{1}{N}$.
Assume that the first request belongs to family j. We find out the expected value of the size of the first scheduler batch.
For the batch to have size exactly x:

- The first $x-1$ requests after the initial one must also be of family j.
- The x-th request must be of a different family.

The probability that the batch is of size x is given by:

$$\Pr[X = x] = \left(\frac{1}{N}\right)^{x-1} \cdot \frac{N-1}{N}, \quad x \geq 1.$$

Greedy First Come First Serve - Appendix - Expected Batch Size



Let X denote the size of the first batch. Its expected value is:

$$\mathbb{E}[X] = \sum_{x=1}^{\infty} x \Pr[X = x] = \sum_{x=1}^{\infty} x \left(\frac{1}{N}\right)^{x-1} \frac{N-1}{N}.$$

Using the identity

$$\sum_{x=1}^{\infty} x r^{x-1} = \frac{1}{(1-r)^2}, \quad |r| < 1,$$

with $r = \frac{1}{N}$, we obtain:

$$\mathbb{E}[X] = \frac{N-1}{N} \cdot \frac{1}{\left(1 - \frac{1}{N}\right)^2} = \frac{N}{N-1}.$$

$$\boxed{\mathbb{E}[\text{batch size}] = \frac{N}{N-1}}$$

For example, when we have 2 families, the expected batch size would be just 2.

Family ID Based



- If we can set priority values for each of the request, then why can't we set the family to which they belong to?
- We have separate queues for each family
- All the individual queues operate in a first come first serve manner
- If there is a request of family ID j already present in the scheduler batch, then we try to add all the requests of that family in the same batch (till we hit `max_batch_size`). If there is no request of that family available, then we stick to the current batch (no more requests added)
- So, the batch size is no longer static - it is dynamically set in accordance with the workload

Radix Tree Based



- Key idea: Instead of treating each request independently, we exploit shared token prefixes across requests
- Each request is represented as a path of tokens in a radix tree
- Internal nodes correspond to shared token substrings
- The scheduler maintains a working set of active nodes (tokens already materialized / cached)
- Each waiting request has a cost = number of tokens missing from the working set

Radix Tree Based



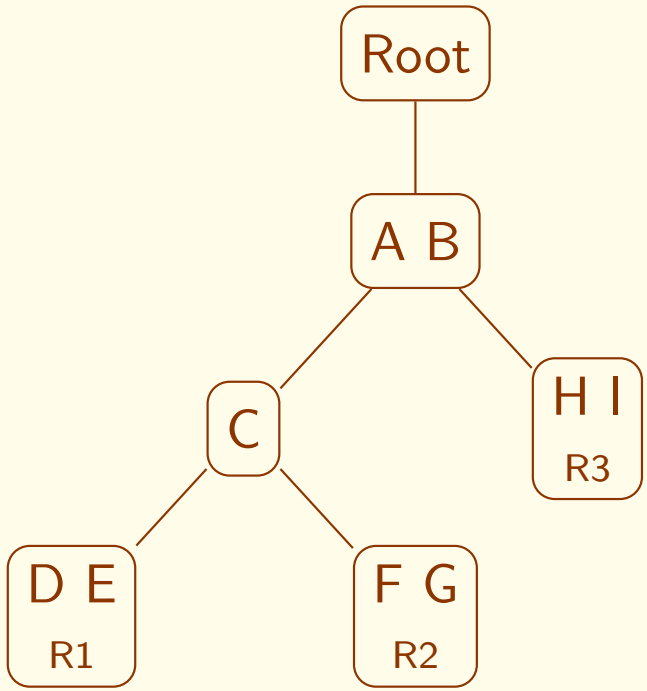
To schedule the next request:

- We start from leaves already in the working set
- Traverse upwards toward the root
- At each node, we look for waiting requests passing through it
- We select the request with minimum missing-token cost

Activating a request:

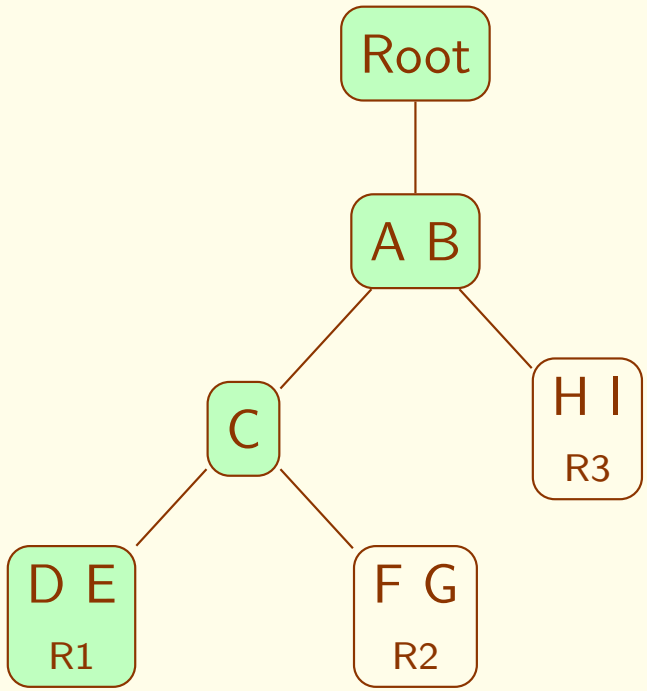
- Adds its path to the working set
- Future requests that share prefixes become cheaper
- This dynamically forms prefix-coherent batches, minimizing recomputation

Radix Tree Based - Radix Tree Construction



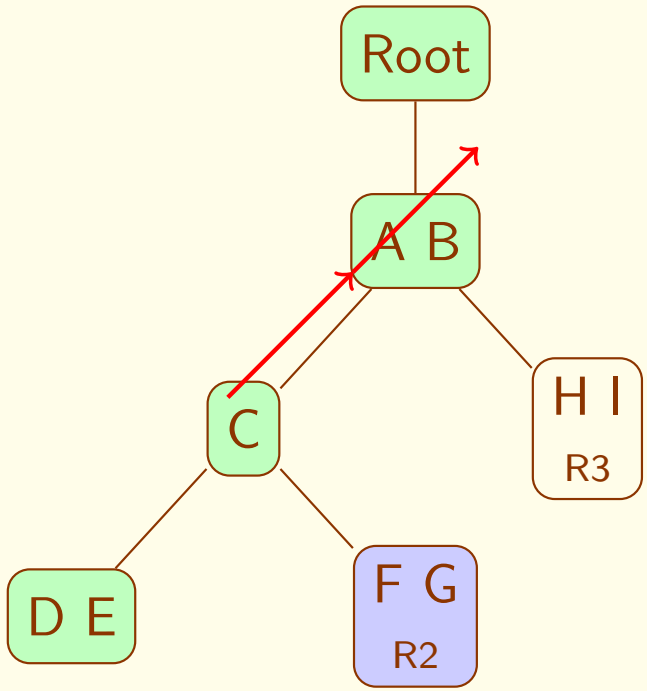
Each leaf corresponds to a request. Internal nodes represent shared token prefixes.

Radix Tree Based - Working Set After Activation



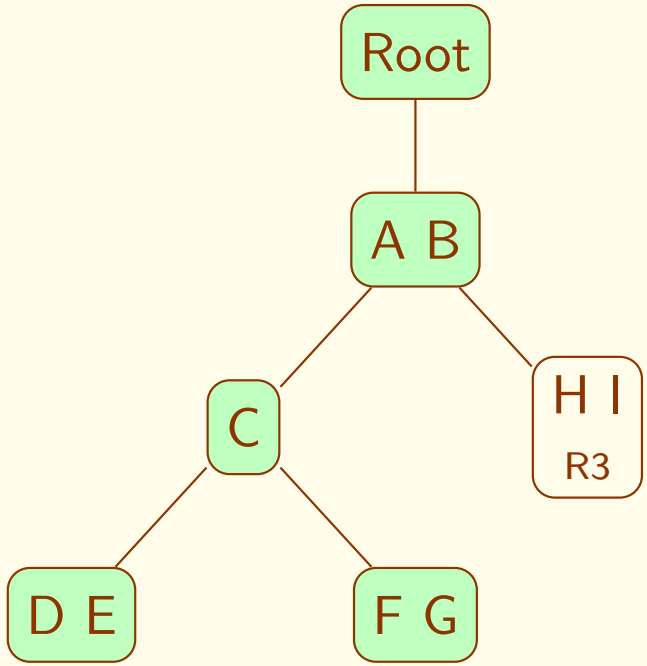
Green nodes indicate the working set after activating R1.

Radix Tree Based - Finding the Best Request



Traversal starts from working-set leaves and moves upward. R2 is selected due to maximum prefix overlap.

Radix Tree Based - Dynamic Batch Growth



Batch size grows dynamically as long as marginal cost is low.

Radix Tree Based - Issues



The above approach has a few issues:

- Adding a request is quite costly - there might be splitting/merging of nodes. Although finding the best request is quite efficient
- Each request may contain thousands of tokens, so the tree depth might get quite large too
- We cannot have a parallel thread/process handling the radix tree, because maintaining sync is difficult. What if we removed a request, from our waiting queue, but it takes some time to remove it from the radix tree?

Chunked Hash Tree Based



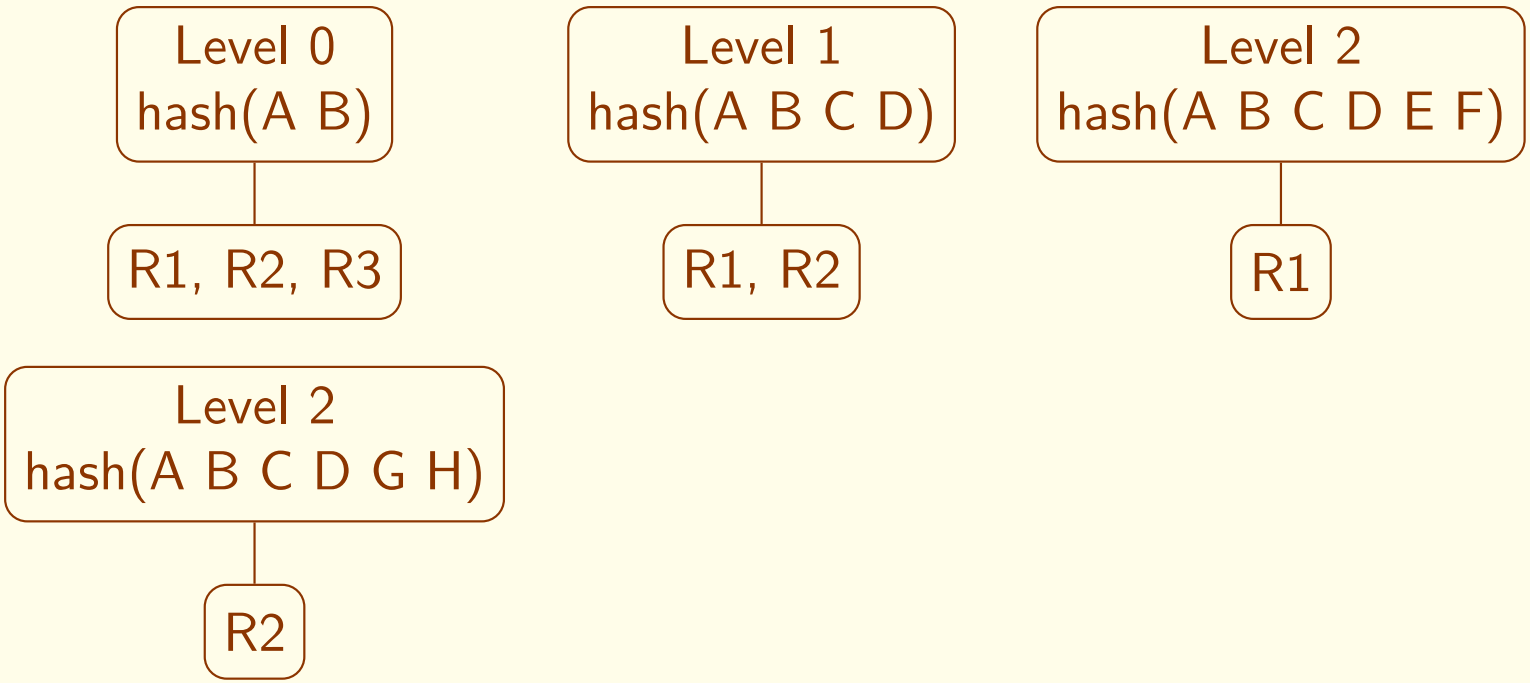
- Requests are represented using **hashed cumulative prefixes**
- Tokens are divided into fixed-size chunks
- Each level corresponds to `hash(tokens[0:k])`
- No explicit tree structure or node splitting
- Much faster than radix trees for large workloads

Chunked Hash Tree Based - Working Set



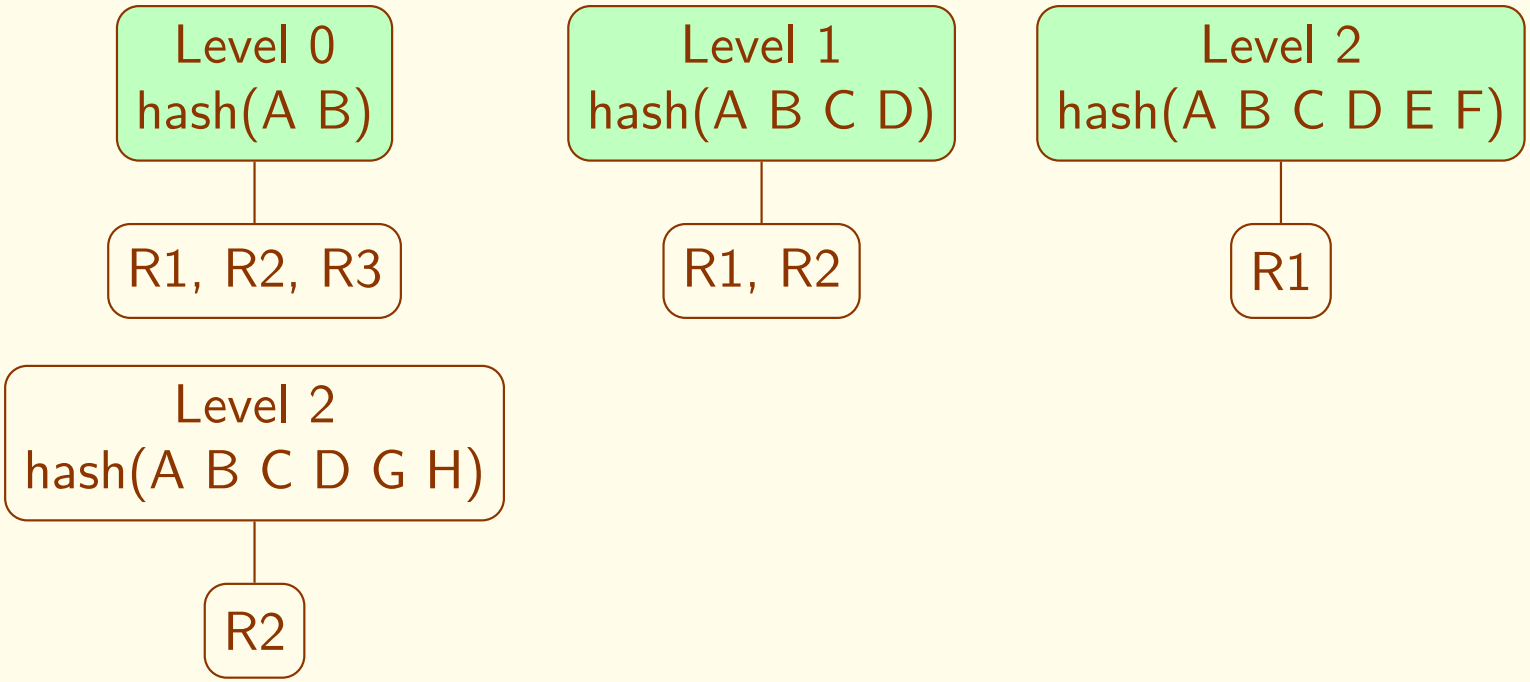
- Working set = set of active **(level, hash)** pairs
- Each active request contributes all its prefix hashes
- Request cost = number of missing prefix hashes
- Scheduling prefers requests with minimal missing chunks
- Enables dynamic batch growth similar to radix trees

Chunked Hash Tree Based - Insertion



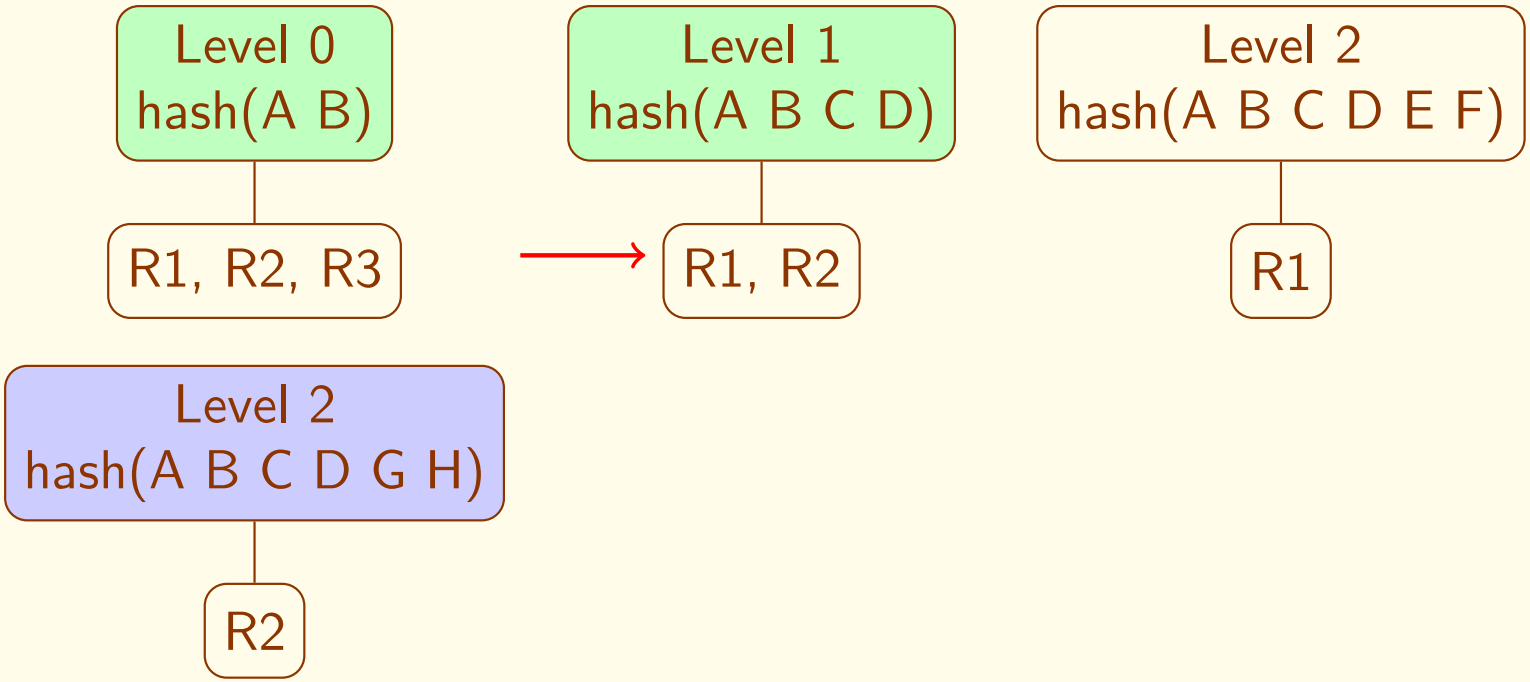
Each level stores hashes of cumulative token prefixes.

Chunked Hash Tree Based - Working Set



Green boxes indicate prefix hashes in the working set.

Chunked Hash Tree Based - Best Request Selection



R2 is selected since it has maximum prefix overlap with the working set.



Thank You